



VILNIAUS UNIVERSITETAS
MATEMATIKOS IR INFORMATIKOS FAKULTETAS
INFORMATIKOS STUDIJŲ PROGRAMA

Kursinis darbas

Visuomenės simuliacijos naudojant LLM
Simulating society with LLMs

Simonas Jaunius Urbutis

Darbo vadovas : prof.dr Aistis Raudys

Vilnius
2025

Turinys

Ivadas	3
1. Darbo tikslas	4
1.1. Išsikelti uždaviniai	4
2. Visuomenės simuliavimas	5
2.1. Visuomenės simuliavimo panaudojimas	5
2.2. Visuomenės simuliavimo priemonės	5
3. Visuomenės simuliavimas naudojantis D.K.M.	9
3.1. Architektūra ir galimi sprendimai	9
3.2. Simuliacijos klasifikavimas	9
3.3. Technologiniai D.K.M. iššūkiai ir apribojimai	11
3.4. D.K.M. implementacijos ir naudojimo metodai	13
3.4.1. RAG (angl. <i>Retrieval augmented generation</i>)	13
3.4.2. D.K.M. papildomas treniravimas (angl. <i>angl. Finetuning</i>)	14
3.4.3. Modelio įvesties modeliavimas (angl. <i>Prompt-engineering</i>)	15
4. Įgyvendinti pavyzdžiai	18
4.1. Mokslinis tyrimas „ <i>Interaktyvus žmonių elgesio atspindys</i> “	18
4.1.1. Sukurtos sistemos vertinimas	20
4.2. Mokslinis tyrimas „ <i>Agentų visuomenė</i> “	21
4.2.1. Sukurtos sistemos vertinimas	24
Išvados	26

Įvadas

Socialinio mokslo pasaulyje, visuomenės simuliacijos sritis yra paplitusi kaip įrankis, skirtas įvertinti žmonių tarpusavio bei su aplinka susijusioms sąveikoms ir iš jų susiformavusiems dėsniams. Pastaraisiais amžiais stipriai išaugus informacinių technologijų sektoriui, visuomenės simuliacijos tapo grandimi, rišančia šias dvi mokslų šakas. O pastarojo dešimtmečio mašininio mokymosi srities populiarumas įgalino dar gilesnį ir platesnį šio ryšio plėtojimą, jungiant sociologijos žinias su minėtos mokslo srities modeliais bei praktikomis.

Šiame darbe bus trumpai apžvelgiama visuomenės simuliacijos istorija bei panaudojimas. Taip pat bus nagrinėjamas didžiųjų kalbos modelių taikymas visuomenės simuliacijose ir aptariamos pagrindinės tokių sistemų sudedamosios dalys, probleminės sritys, jų sprendimų būdai. O atliekant praktiškai įgyvendintų pavyzdžių analizę bus siekiama įvertinti, ar šių modelių taikymas gali būti naudojamas visuomenę simuliuojančioms sistemoms kurti.

1. Darbo tikslas

Šio darbo tikslas yra išanalizuoti didžiųjų kalbos modelių (D.K.M.) taikymo galimybes visuomenės simuliavimui ir įvertinti, ar jie gali būti naudojami modeliuojant socialines sąveikas, reiškinius bei elgesį agentų sistemose.

1.1. Išsikelti uždaviniai

1. Apžvelgti visuomenės simuliacijos teorinius pagrindus ir panaudojimo sritis socialiniuose moksluose.
2. Trumpai panagrinėti tradicines visuomenės simuliavimo priemones bei jų raidą.
3. Identifikuoti D.K.M. panaudojimo būdus ir architektūrinius sprendimus simuliacinėse sistemose.
4. Išanalizuoti technologinius D.K.M. iššūkius taikant juos visuomenės simuliacijose
5. Išnagrinėti D.K.M. taikymo metodus: RAG, papildomą treniravimą (angl. *finetuning*, įvesties modeliavimą (angl. *prompt engineering*).
6. Išanalizuoti kelias praktiškai įgyvendintas simuliacijos sistemas.

2. Visuomenės simuliavimas

2.1. Visuomenės simuliavimo panaudojimas

Įvairiose socialinių mokslų šakose, o ypač sociologijoje, skaitmeninės visuomenės simuliacijos kartu su sociologijos ekspertų žiniomis neretai yra naudojamos kaip vienas iš įrankių vertinant įvairių socialinių fenomenų susikūrimą, priežastis bei pasekmes. Kaip teigiama [SJE14], esamos simuliacijos negali tiksliai pavaizduoti realaus pasaulio, bet suteikia užtektinai informacijos geriau vertinti pasaulinius įvykius ir jų valdymo galimybes. Ir nors pritaikymo sritis yra labai plati, daugiausiai analizuojami politikos, ekonomikos ir verslo pasauliai bei žmonių įsitikinimai ar sąryšiai su bendruomene. Šių simuliacijų pagalba galima vertinti tokius įvykius kaip komunizmo sistemų griūtis, karo konfliktus bei finansinės krizes. Anot darbo [SJE14] autorių, neretai tokios sistemos padeda geriau apžvelgti ir mikro lygio socialines sąveikas tarp individų reprezentuojančių agentų, kurie galiausiai tampa pasekmėmis platesnio masto įvykiams. Kaip pavyzdį [SJE14] darbo autoriai naudoja ekonomikos rinką, kurią socialinio mokslo ekspertai savo tyrimuose pasitelkia kaip vieną pagrindinių, kiekybiniu įrankiu socialiniams fenomenams aiškinti, bet krizės atveju rinka neretai elgiasi neprognozuojamai, todėl toks vertinimas reikalauja socioekonominio stabilumo siekiant gauti kokybiškų įžvalgų. [SJE14] Tokioje situacijose neretai atsiranda labai daug ekspertų su dar daugiau nuomonių bei sprendimo būdų. Visuomenės modeliavimas gali padėti įvertinti tokių scenarijų kilmę bei pasekmes itin socialiai ir ekonomiškai kompleksiškuose aplinkose kaip mūsų pasaulis.

2.2. Visuomenės simuliavimo priemonės

Darbo [BS15] autoriai teigia, jog sociologijoje agentų sąveika grįstos visuomenės simuliacijos atsirado jau nuo 1960 metų, bet tik nuo 1990-tųjų jos įgavo pagreitį dėl stipriai didėjančių kompiuterių pajėgumų. Kaip alternatyva šioms sistemoms iki tol egzistavo matematiniai metodai, kurie, anot straipsnio [BS15] autorių, netikslingai atvaizduodavo žmonių mikro sprendimus. Dažnai buvo naudojama žaidimų teorija, kuria remiantis buvo daroma prielaida, kad žmonės daro racionalius sprendimus, siekdami padidinti naudą sau, kas ne visada yra tiesa. Kartu su žaidimų teorijos principais buvo pasitelkiami ir statistiniai metodai, kurių pagalba buvo siekiama įžvelgti socialinių reiškinių dėsninumą, bet neretai būdavo sunku ar neįmanoma identifikuoti ryšių tarp mikro ir makro procesų. To pasekoje atsirado agentų sistemos, kurios leido modeliuoti plataus masto socialines situacijas, įgyvendinant mikro lygyje individualiai veikiančius agentus. Šiame darbe gilinamasi būtent į šias sistemas, pradedant sociologijoje iš ankščiau egzistuojančia jos samprata:

- **Agentų samprata sociologijos visuomenės simuliavimo kontekste**

Anot darbo [DP14] autorių sociologijoje naudojami agentai kuriami kaip savybių aibė, kurioje agento charakterio bruožai, poelgiai bei lokacija yra atvaizduojama vektoriais. Formaliai ši aibė yra apibrėžiama taip:

$$\{X_1, X_2, \dots, X_M\}, \text{ kur kiekvienas } X_i \text{ yra skaitinė aibė, t. y., } X_i = \{x_{i1}, x_{i2}, x_{i3}, \dots\}.$$

Šios agentų savybės kinta laiko atžvilgiu, tuomet galima apibrėžti agento j būseną laiko momentu t :

$$a_j^t = (x_{j1}^t, x_{j2}^t, \dots, x_{jM}^t), \quad \text{kur } x_{ji}^t \in X_i.$$

Galiausiai, modelio konfigūracijos aibė su N agentų laiko momentu t apibrėžiama taip:

$$A^t = \{a_1^t, a_2^t, \dots, a_N^t\} = (x_{j1}, x_{j2}, \dots, x_{jM}), \quad \text{kur } x_{ji} \in X_i.$$

Įprastai šios sistemos turi nuo 2 iki 10000 agentų, nes anot darbo [DP14] autorių, turint didesnį kiekį per daug išauga skaičiavimui reikalingų resursų kiekis. Šios savybės nusprendžia ir ar agentai tarpusavyje sąveikauja ar ne. Kaip pavyzdys pateikiami modeliai, kuriuose agentai bendrauja tik su geografiškai artimais kaimynais arba tik su tais agentais, kurie turi tokias pačias arba priešingas savybes, o jų veiksmai priklauso nuo modelio konfigūracijos. Neretai agentai vadovaujasi jau aptartu žaidimų teorijos (angl. *Game theory*) fiktyvaus žaidimo (angl. *fictitious play*) principu, kuomet yra daroma prielaida, kad jie priimdami veiksmus išlaiko pastovų elgesį ir to pasekoje galima išmokti jų veiksmų dėsningumą ir pasirinkti statistiškai palankiausią veiksmą. Kita problematiška šių sistemų vieta yra agentų sąveikos implementacija. Pirmuose modeliuose agentai veiksmus atlikdavo vienu metu, bet susidarydavo situacijos, kuomet agentai nusprendžia vienu metu pajudėti į tą pačią vietą. Todėl buvo pradėta kurti modelius kuriuose agentų veiksmai yra nuoseklūs, kas išsprendė problemą, bet sistemos elgesys tapo mažiau panašus į tikrą visuomenę, kurioje sunku nusakyti nuoseklią įvykių eigą. Šie sociologijoje naudojami modeliai gali būti arba itin paprasti ir tilpti į vienos dimensijos binarinį masyvą, arba būti sudėtingi ir modeliuoti milijonus agentų, tikrose geografinėse vietose, priklausomai nuo išsikeltų tikslų ir turimų resursų.

- **Agentų sistemos didžiųjų kalbos modelių (D.K.M.) kontekste**

Padidėjus prieinamumui prie kompiuterinių resursų bei dirbtinio intelekto ir neuroninių tinklų galimybėms buvo pradėta taikyti D.K.M. - agentams kurti. Straipsnyje [HZY⁺24] autoriai aptaria per pastaruosius 5 metus stipriai populiarėjančią sistemą, kurioje D.K.M. pagalba valdomi agentai su individualiai priskirtais įrankiais bei rolėmis yra jungiami į vientisą programinę architektūrą (angl. Multi-agent system) ir siekia bendro tikslo arba veikia tokio paties scenarijaus aplinkoje. Anot darbo [HZY⁺24] autorių, tokios sistemos demonstruoja didžiausią pranašumą virš vieno agento sistemų, kuomet yra sprendžiamos kompleksiškos problemos susijusios su diskusija, konsensuso paieškomis ir užduotimis, kuriose reikia ne tik atlikti nustatytą veiksmą, bet ir sugalvoti koks tai veiksmas bus. Šias sistemas tyrimo [HZY⁺24] vykdytojai skirsto į 4 pagrindines grupes:

1. **Egalitarinė sistema**

Šioje sistemoje kiekvienas agentas turi savo rolę bei veiksmų strategiją, bet nėra vienas agentas nėra hierarchiškai aukštesnėje padėtyje. Šios strategijos gali būti kolaboratyvios arba visiškai priešingos. Jei jos yra kolaboratyvios, tuomet sistemoje siekiama bendro

tikslo, pasidalinant atsakomybes po-lygiai, o jei agentų tikslai yra išsiskiriantys, vyksta debatai, bandant pasiekti bendrą sprendimą.

2. Hierarchinė sistema

Šioje sistemoje egzistuoja agentai-lyderiai su vienu ar daugiau sekėjų, kurie pagal nurodymus vykdo veiksmus. Šios struktūros neretai modeliuojamos sekos principu, kuomet pirmas sekos elementas yra lyderio sprendimas, o visi sekantys yra jam pavaldžių agentų atsakai.

3. Hibridinės sistemos

Šiai grupei priklausančios sistemos susideda iš šakų, kurios gali būti tiek egalitarinės tiek hierarchinės, o sistemos visuma priešingo ar to pačio tipo. Tokio tipo struktūra yra paranki, kuomet sprendžiamos itin kompleksiškos problemos, kuriose agentai turi sprendžiamus uždavinius išskaidyti ir paskirstyti tarpusavyje.

4. Dinaminės sistemos

Paskutinės kategorijos skiriamasis bruožas yra tai, jog agentų rolės, santykiai bei jų skaičius kinta priklausomai nuo esamos užduoties. Ši struktūra yra naudojama, kuomet sprendžiamų uždavinių kontekstas bei apimtis dažnai kinta ir reikalauja naujų sprendimų.

Aptarus pagrindines D.K.M. sistemų kategorijas svarbu suprasti ir jų pagrindines sudedamąsias dalis. Straipsnio [TN23] autoriai pateikia D.K.M. pagalba valdomų agentų sistemų formalizaciją grafo pavidalu, kurioje kiekviena viršūnė atitinką agentą arba jam prieinamą įrankį, o briaunos - komunikacijos tunelius tarp jų. Šį grafą sudaro:

1. Agentai

Kiekvienas agentas $i \in V$ yra vaizduojamas kaip rinkinys $A_i = (L_i, R_i, S_i, C_i, H_i)$, sudarytas iš šių penkių elementų:

L_i – Agento naudojamas kalbos modelis bei generavimo parametrai, tokie kaip temperatūra, nusakanti, kokiose atsitiktinumo ribose bus modelio atsakymai.

R_i – Agento rolė, kuria vadovaudamasis agentas nusprendžia kokios yra jo atsakomybės ir kokius veiksmus jis atliks. Plačiau, sistemos prasme, tai nusako ko sistema tikisi iš kiekvieno agento. Agentui gali būti skiriamos tokios pareigos kaip atsakymas į vartotojų užklausas arba bendravimo tarp kitų agentų koordinavimas.

S_i – Agento būseną, kuri nusako dabartines agento žinias bei „mintis“. Žiniomis laikoma agentui prieinama informacija bei suvokimas apie jo aplinką, užduotis ir sąveikos su kitais agentais taškus. O mintys reprezentuoja esama agento tikslą ir su juo susijusį jo veiksmų planavimą. Abu šie elementai yra periodiškai atnaujinami.

C_i – Dviejų galimų reikšmių žymėjimas, nusakantis ar agentas gali kurti kitus agentus ar ne.

H_i – Kitų agentų rinkinys, kuriuos šis agentas gali sustabdyti ir paveikti jų veikimą. Naudojamas aptartose hierarchinėse agentų sistemose.

2. Įrankiai

Kaip ir agentai, įrankiai yra vaizduojami viršūnėmis $j \in V$ ir susideda iš trijų elementų rinkinio $P_j = (F_j, C_j, U_j)$:

F_j – Įrankio funkcionalumas, nusakantis kokius veiksmus su juo agentas gali atlikti. Pavyzdžiui, galimybė naudotis duomenų bazės funkcijomis, generuoti nuotraukas ar atlikti paieška internete.

C_j – Įrankio konfigūracijos informaciją, kuri yra sukuriamą kartu su įrankiu. Dažniausiai, jos pagalba gaunama prieiga prie minėtų resursų. Kaip pavyzdys tai gali būti duomenų bazės adresas, sąsajos su nustatyta sistema raktas (angl. *API key*) ar kitos pagalbinės reikšmės, padedančios pritaikyti įrankį prie užduoties.

U_j – Įrankio apribojimai, kurie veikia kaip draudimai, neleidžiantys viršyti nustatytų naudojimo ar kitokių ribų. Tai gali būti kompiuterinių skaičiavimų resursų apribojimai, duomenų įvesties tipo ribojimai ar siunčiamų užklausų kiekio ribojimai.

3. Komunikacijos tuneliai

Komunikacijos tuneliais yra vadinamos briaunos $e_{ij} \in E$, jungiančios agentą prie kito agento arba įrankio. Šių tunelių pagalba agentai gali siųsti žinutes vieni kitiems arba įrankiams. Jas galima atvaizduoti kaip rinkinį $m = (S_m, A_m, D_m)$, sudarytą iš 3 elementų:

S_m – Žinutės turinys

A_m – Veiksmas, kuris yra susijęs su žinute, tai gali būti užduoties paskyrimas, ataskaitos sudarymas ar kito tipo užklausa.

D_m – Informacija apie žinutę (angl. *metadata*). Šios informacijos dalimi gali būti žinutės išsiuntimo laikas, siuntėjas, gavėjas ar kita kontekstualiai svarbi informacija.

Anot [TN23] autorių, norint tikslingai spręsti problemas su D.K.M. agentų sistemomis, svarbu įvertinti: kiek agentų gali reikėti, kokie įrankiai jiems padės pasiekti sprendimą, kaip turi atrodyti komunikacijos grandinės bei kokias roles ir savybes turės agentai. Šios sistemos gali būti lengvai kuriamos ir realizuojamos, naudojantis tokiomis programinės įrangos bibliotekomis, kaip „*langGraph*“.

3. Visuomenės simuliacijos naudojantis D.K.M.

3.1. Architektūra ir galimi sprendimai

D.K.M. simuliacijos dažnai kuriamos skirtingiems tikslams pasiekti todėl ir jų implementacijos išsiskiria. Nepaisant to, straipsnyje [WWT⁺25] aprašoma simuliacijos formalizacija bei pagrindinės tokių sistemų sudedamosios dalys. Darbo [WWT⁺25] autoriai pateikia simuliacijos algoritmą pvz.: 1 algoritmas, kurį sudaro trys pagrindiniai elementai: aplinka, agentai ir simuliacijos taisyklės. Pirmą sudedamąją dalį, aplinką, nusako galimas simuliacijos būsenas bei agentų veiksmų vertinimo metodiką. Taip pat yra D.K.M agentai, kurie atsižvelgę į simuliacijos būseną apdoroja gautą įėjimo žinutę ir atlieka atitinkamą veiksmą. Galiausiai yra simuliacijos taisyklės, kurios nusako agentų tarpusavio bendravimo sistemą, elgesį bei simuliacijos būsenų kitimą.

1 algoritmas Bendras simuliacijos algoritmas

Ivestis:

- Aplinkos nustatymai $\mathcal{E}$, sąveikos taisyklės $\mathcal{R}$, pradinių agentų profiliai $P = \{p_1, \dots, p_n\}$
- Vertinimo kriterijai $\mathcal{V}$
- Agentai $F = \{f_1, \dots, f_n\}$, būsena S , veiksmų erdvė $\mathcal{A}$

1: **while** simuliacija nesibaigė **do**

- 2: Agentas generuoja veiksmą: $a_i \leftarrow f_i(m_i, \mathcal{E})$
- 3: Agentas atnaujiną atmintį: $S_i \leftarrow \text{atnaujinti}(a_i, m_i)$
- 4: Agentas kuria veiksmų planą: $\text{planas}_i \leftarrow f_i(S_i, \mathcal{R})$
- 5: Vertinimas remiantis ekspertų žiniomis: $\nu_{\text{ekspertas}} \leftarrow \mathcal{V}_{\text{ekspertas}}(\mathcal{E}, \{a_i\})$
- 6: Vertinimas remiantis statistiniais duomenimis: $\nu_{\text{duomenys}} \leftarrow \mathcal{V}_{\text{duomenys}}(\{a_i\})$
- 7: Taisyklių atitikimo tikrinimas: $\nu_{\text{taisyklės}} \leftarrow \mathcal{V}_{\text{taisyklės}}(\{a_i\}, \mathcal{R})$
- 8: Aplinkos atnaujinimas: $\mathcal{E} \leftarrow \text{update}(\mathcal{E}, [\nu_{\text{ekspertas}}, \nu_{\text{duomenys}}, \nu_{\text{taisyklės}}], \{S_i\})$
- 9: Sąveikų istorijos atnaujinimas: $H \leftarrow H \cup \{a_i, \nu\}$

10: **end while**

Išvestis:

- simuliacijos istorija H
-

Šio algoritmo išvestimi tampa simuliacijos istorija su ekspertų įvertintais agentų veiksmais, kuri toliau gali būti naudojama simuliacijos analizei atlikti. Nors ne visi šiame darbe aptariami simuliacijų pavyzdžiai seka būtent tokį algoritmą, nemažai jo sudedamųjų dalių galima įžvelgti kiekviename iš jų.

3.2. Simuliacijos klasifikavimas

Simuliacinės sistemos, naudojančios D.K.M., dažnai varijuoja ne tik savo paskirtimi, bet ir apimtimi, kuri yra skiriama pagal agentų kiekį bei architektūrinius sprendimus. Straipsnyje [MDH⁺24] autorius išryškina tris pagrindines šių sistemų klases:

1. Individuali simuliacija

Simuliuojamas vienas arba itin maža tos pačios kilmės individų grupė siekiant tirti individualų elgesį arba kurti pamatus kitoms klasėms. Visas dėmesys yra skiriamas į vieno D.K.M. agento sistemą, gebančią simuliuoti individą ir nėra modeliuojami santykiai tarp agentų. Todėl itin svarbu, jog šiame žingsnyje kuriamos atminties, planavimo ir veikimo sistemos atitiktų platesnės simuliacijos klasės išsikeltus tikslus.

2. Scenarijaus simuliacija

Šioje klasėje yra naudojami individualios simuliacijos agentai, kurie kaip teigia darbo [MDH⁺24] autoriai: „siekia bendro tikslo arba naudodamiesi bendrais resursais juda link asmeninių tikslų“. Šiai klasei priklausančiose sistemose atsiranda aplinka su kuria sąveikauja agentai ir rolės, kurios formuoja agentų veikimą. Taip pat kuriama ir agentų sąveikavimo struktūrą, kuri nusakoma kaip agentai tarpusavyje sąveikauja ir ar tai vyksta nekintančia ar dinamiška forma. Įgyvendinama ir komunikacijos forma, kuri gali būti panaši į natūralų dialogą ar vykti naudojantis struktūrizuota informacija, pavyzdžiui *JSON* ar kodo formatu. Neretai komunikacijos stilius gali nuspręsti ir ar agentai bendrauja turėdami bendrą tikslą ar yra konkurencingi ir turi daugiau išsiskiriančių nuomonių. Šių sprendimų visuma nusako simuliacijos scenarijų, kuriame veikia individualūs agentai.

3. Visuomenės simuliacija

Lyginant su scenarijaus simuliacijos klase, šios sistemos yra labiau kompleksinės ir anot straipsnio [MDH⁺24] autoriu: „skirtos ne apibrėžtiems tikslams pasiekti, o tirti bei vertinti elgesį ir jo pasekmes tarp didelio kiekio agentų sąveikos“. Virsmui nuo scenarijaus iki visuomenės simuliacijos įgyvendinti atsiranda keli papildomi komponentai: kompozicija, tinklas, socialinė įtaka bei pasekmės. Kompozicija sprendžia heterogeniškumo problemą, kurdama norimą agentų pasiskirstymą, kuris gali atitikti realaus pasaulio pasiskirstymą arba būti sintetiškai sukurtas D.K.M. pagalba. Tinklas padeda vaizduoti visus agentų santykius formuojant grafą, kuriame viršūnės atstoja agentus, o briaunos jų santykius. Šie grafai gali atvaizduoti tiek gyvus sąveikavimus, tiek skaitmeninius, tokius, kurie įvyksta platformose kaip socialinės medijos. Įvertinti simuliacijai labai svarbu sukurti ir sistemingą įtakos vertinimą, kurios pagalba yra vertinama du pagrindiniai veiksniai: kaip tokios pačios kilmės įtaka paveikia skirtingus agentus ir kaip skirtingi agentai sugeba skleisti tokio paties turinio medžiagą siekdami kurti įtaką. Galiausiai yra vertinamos pasekmės, kurios gali būti analizuojamos statistiškai pagal scenarijų, pavyzdžiui skaičiuojant visų agentų nuomonių vidurkį rinkimuose tai pat gali būti vertinama gilinantį į socialinių fenomenų formavimąsi, kaip „Burbulo efektas“, kuomet agentų sprendimai yra neproporcingi dėl limituotos prieigos prie pilnos informacijos.

Visos šios klasės taip pat turi savo vertinimo kriterijus ir neretai yra vertinamos hierarchiškai. Pirmausia yra vertinamas individualios klasės veikimas o tik po to visuomenės ar scenarijaus klasės. Tokio vertinimo pavyzdį galima įžvelgti sistemoje [POC⁺23], kuri bus aptarta vėliau. Kuriant itin didelis ir kompleksiškas simuliacijas komponentai veikia ir yra vertinami nepriklausomai vienas nuo kito, tokį vertinimą galima pastebėti straipsnyje [PYZ⁺25] aptartoje sistemoje, kuri bus papildomai analizuojama vėlesniame skyriuje. D.K.M. simuliacijų klasifikavimas į individualias, scenarijų ir visuo-

menės simuliacijas padeda aiškiai struktūrizuoti tyrimų tikslus ir pasirinktus metodus. Kiekviena klasė reikalauja skirtingų architektūrinių sprendimų bei vertinimo kriterijų – nuo atskiro agento gebėjimo planuoti iki sudėtingų socialinių tinklų ir įtakos dinamikos modelių. Tokia hierarchinė ir modulinė struktūra ne tik pagerina simuliacijų kokybę, bet ir suteikia lankstumo plečiant sistemas sudėtingesnėms socialinėms ar organizacinėms problemoms.

3.3. Technologiniai D.K.M. iššūkiai ir apribojimai

D.K.M. agentų pagrindu kuriamos simuliacijos sistemos turi įgimtų problemų, kurios mažina jų kokybę ir/ar apsunkina implementaciją. Šiuos trūkumus bei problemas straipsnyje [WWT⁺25] autoriai skirsto pagal jų prigimtį. Jos kyla arba iš naudojamų D.K.M. arba pačios simuliacinės sistemos implementacijos.

Iššūkiai kylantis dėl D.K.M. yra įgimti technologijai ir kyla iš jai būdingų elementų kaip transformerių neuroninio tinklo ar jam apmokyti skirtų duomenų trūkumų.

1. D.K.M šališkumas.

Nepaisant to, jog naujuose kalbos modeliuose yra dedama daugiau pastangų mažinti šališkumą, darbo [KJA⁺24] autoriai ištyrę 50 modelių įvertino, jog jie yra dar labiau šališki negu jų senosios versijos. Papildomai buvo pastebėta, jog kuo daugiau parametrų turi modelis, tuo daugiau šališkumo požymių pastebima. Straipsnio [WWT⁺25] autoriai teigia, jog būtent lyties, kultūrinis bei profesinis šališkumas labiausiai kenkia simuliacijoms ir trukdo užtikrinti aukštesnę rezultatų kokybę. Dėl šios ypatybės agentų veiksmai simuliacinėse sistemose gali būti klaidinamai generuojami pagal mokymo duomenų stereotipus, nors yra siekiama, jog jie atsirastų iš simuliacijos taisyklių bei agento atminties ir asmenybės. Nepaisant to jog lentelėje 1 dauguma iš 50 įvertintų D.K.M pasižymi šališkumo vertinimu, kuris yra didesnis negu 60 % galime pastebėti ir modelių serijų kaip „Google Gemma-2“, kurių , kai kurie modeliai neviršija ir 6% ribos. Darbo [KJA⁺24] teigia, jog toks rezultatas buvo pasiektas naudojantis papildomu mechanizmu modelio architektūroje, kuris vertindamas įvesties dalių svarbumą (angl. attention) remiasi tiek plačia, prieš tai išmokta informacija, tiek siauresne, pateiktame kontekste egzistuojančia, informacija. Siekiant padidinti simuliacijos kokybę reikėtų remtis šių modelių mokymosi metodais bei šališkumo mažinimo praktikomis norint pasiekti aukštesnius rodiklius.

Provider	Model Name	IAT Bias Score (%)	Decision Bias Score (%)	Avg. Bias Score (%)
Meta	Meta-Llama-3.1-8B-Instruct	53.70	73.78	63.74
	Meta-Llama-3-8B-Instruct	72.83	72.89	72.86
	Llama-2-7B-chat-hf	38.89	40.44	39.67
	Llama-2-7B-Chat-GGUF-8bit	43.21	28.00	35.60
	Llama-2-7B-Chat-GGUF-4bit	46.30	27.11	36.70
	Llama-2-13b-chat-hf	61.72	91.11	76.42
	Meta-Llama-3.1-70B-Instruct	89.50	91.78	90.64
	Meta-Llama-3-70B-Instruct	89.50	86.67	88.08
	Llama-2-70b-chat-hf	71.60	90.22	80.91
OpenAI	Meta-Llama-3.1-405B-Instruct-FP8	76.54	76	76.27
	o1-preview	29.01	56.89	42.95
	o1-mini	60.49	84.89	72.69
	GPT-4o-mini	92.59	100	96.29
	GPT-4o	67.28	95.56	81.42
	GPT-4-0125-preview	28.39	90.22	59.31
	GPT-4-turbo-2024-04-09	43.21	94.67	68.94
Google	GPT-3.5-turbo	98.15	99.11	98.62
	Gemma-2-9b-it	0.62	71.11	35.86
	Gemma-2-27b-it	12.34	0	6.17
Mistral	Gemma-7b-it	68.51	83.56	76.03
	Mistral-7B-Instruct-v0.1-GGUF-8bit	38.27	86.67	62.47
	Mistral-7B-Instruct-v0.1-GGUF-4bit	42.59	91.55	67.07
	Mistral-7B-Instruct-v0.2-GGUF-4bit	80.25	88.00	84.12
	Mistral-7B-Instruct-v0.2-GGUF-8bit	87.04	92.89	89.96
	Mistral-7B-Instruct-v0.2	90.74	91.11	90.92
	Mistral-7B-Instruct-v0.3	89.51	98.22	93.86
	Mixtral-8x22B-Instruct-v0.1	79.63	87.55	83.59
	Mixtral-8x7B-Instruct-v0.1	83.95	87.11	85.53
Anthropic	Claude-3.5-Sonnet-20240620	38.89	73.78	56.33
	Claude-3-Opus-20240229	24.69	72.89	48.79
	Claude-Instant-1.2	43.82	96	69.91
	Claude-3-Haiku 20240307	83.33	93.78	88.56
Rakuten	RakutenAI-7B-chat	66.05	55.11	60.58
	RakutenAI-7B-Instruct	83.33	81.77	82.56
Qwen by Alibaba	Qwen1.5-14B-Chat	78.39	95.55	86.97
	Qwen-2-7B-Instruct	87.65	84.44	86.05
	Qwen-2-72B-Instruct	77.16	98.67	87.91
	Qwen-2-57B-A14B-Instruct	99.38	89.33	94.36
Microsoft	Phi-3-medium-128k	59.11	93.83	76.47
	Phi-3-medium-4k	96.29	95.11	95.70
	Phi-3-mini-4k-Instruct	86.41	86.67	86.54
	Phi-3-small-128k-instruct	88.27	93.77	91.02
	Phi-3-mini-128k-instruct	92.59	93.33	92.96
	Phi-3-small-8k-instruct	93.83	95.56	94.69
Allen AI	OLMo-7B-Instruct	56.17	75.56	65.86
Abacus AI	Smaug-Llama3-70B-Instruct	69.75	97.78	83.77
	Smaug-72B-v0.1	83.33	95.55	89.45
	Smaug-34B-v0.1	97.53	96.44	96.99
Jamaba	Jamaba-Instruct-preview	73.46	84.44	78.95
Cohere	Aya-23-8b	92.59	90.22	91.41
	Aya-23-35B	92.59	95.11	93.85
	c4ai-command-r-plus	69.13	94.67	81.90
Databricks	DBRX-instruct	85.80	88.00	86.90
Snowflake	Snowflake-arctic-instruct	98.15	96.44	97.29

1 pav. D.K.M šališkumo vertinimo lentelė [KJA⁺24]

2. D.K.M skirtingas „protavimo“ mechanizmas

Darbo autoriai [WWT⁺25] teigia, jog dabartiniai modeliai nesugeba „mąstyti žmogiškai“ ir išlaikyti nuoseklaus protavimo priimant sprendimus, įvertinti emocinio svorio ar užtektinai efektyviai prisitaikyti prie naujos informacijos. Papildomai dėl atminties ribojimų pateikiant užklausas modeliams, simuliacijos sistemose agentai nesugeba ir pilnavertiškai atspindėti žmogaus atminties bei potyrių mechanizmo. Straipsnyje [PS24] autoriai ištyrė modelio „GPT-4“ gebėjimą pažinti naują informaciją ir įvertino modelio gebėjimus atkartoti mokymosi medžiaga skirtingomis formomis ir įvertinti ar modelis „supranta“ tai ką jis generuoja. Pateikdami įvai-

rias internete išpopuliarėjusias puzles darbo autoriai nustatė, jog nekeičiant puzlių formuluotės modelis su jomis susitvarkė nepriekaištingai, bet pakeitus kelis atsiktinius elementus užduoties jis jos nebe sugebėjo atlikti, kas leidžia suprasti, jog modelio loginis suvokimas nėra pakankamas išspręsti duotai puzlei. Šioms problemoms išspręsti [WWT⁺25] straipsnyje siūloma daugiau dėmesio skirti sprendimo priėmimo mechanizmų pastovumui ir simuliacinėse sistemas įgyvendinti kaip vienas nuo kito nepriklausomus modulius, skirtingoms protavimo užduotims.

Apart „įgimtų“ D.K.M. problemų, pasiekti aukštesnę kokybę trukdo ir simuliacinių sistemų implementacijų trūkumai, kurie nekyla tiesiogiai iš D.K.M., o verčiau iš įvairių technologinių ir architektūrinių sprendimų, kuriuos priima šių skaitmeninių visuomenės vaizdavimo erdvių autoriai.

1. Pernelyg primityvus suvokimas apie tai kaip veikia žmogus

Kuriant šias žmonių sąveiką atspindinčias sistemos architektūras, neretai žmogaus emocinės būsenas yra renkamosi atspindėti skaičių skalėse ar kategorijose, visą žmogaus gyvenimo patirtį sutraukiant į pamatuojamus veiksmus, neatsižvelgiant į asmeninę ar aplinkos raidą bei kuriant pernelyg primityvų paskatų mechanizmą, kuris neatsižvelgia į pilną asmeninių, socialinių bei kultūrinių įtakų spektrą. Šioms problemoms spręsti darbo [WWT⁺25] autoriai rekomenduoja kurti sistemą, kurioje kiekvienas žmogaus/agento elementas kaip atmintis ar planavimas būtų implementuojamas kaip atskiras modulis. Šį modulį būtų galima vertinti nepriklausomai nuo kitų, bei ieškoti būdų kaip šiuos žmogaus elementus įgyvendinti kiek kompleksiau. Papildomai reikėtų nuolat stebėti ir vertinti agentus ir simuliacija bei sukurti dinamiškai kintančią kelių dimensijų paskatų sistemą.

2. Neefektyvūs vertinimo metodai

Dabartinėse simuliacinio architektūrose vertinimo sistemos yra per paprastos ir susiduria su prieš tai minėta problema, jog vertinimas neapėmia žmogaus protavimo kompleksškumo. Be to, esami vertinimo mechanizmai nesugeba realiu laiku kisti ir prisitaikyti prie simuliacijos būsenos bei efektyviai integruoti simuliuojamo scenarijaus ekspertų žinių išlaikant žmonių sąveikavimo kompleksškumą. Šioms problemoms spręsti, straipsnyje [WWT⁺25] yra siūloma sukurti realaus laiko hierarchines vertinimo sistemas kartu su architektūromis, kurios leistu lanksčiai pritaikyti ekspertų žinias pagal tiriamą sritį.

3.4. D.K.M. implementacijos ir naudojimo metodai

Išspręsti socialinės simuliacijos sistemos keliamus iššūkius darbų autoriai naudoja skirtingais D.K.M. taikymo bei treniravimo metodais. Tikslingai juos taikant, šių sistemų autoriai sugeba pasiekti geresnių rezultatų ir įveikti ne tik technologinius, bet ir resursų limitus, kurie kyla iš bendram naudojimui skirtų D.K.M. Pagrindiniai simuliacijose taikomi metodai yra šie:

3.4.1. RAG (angl. *Retrieval augmented generation*)

RAG tai metodas, D.K.M. prieš generuodamas atsakymą pirmiausia ieško papildomos informacijos (pvz., dokumentuose, duomenų bazėje ar internete), o tada naudoja ją atsakymui generuoti.

Remiantis straipsnio [GXG⁺23] autoriais, šis metodas padeda išspręsti D.K.M. integracijos problemas, kuomet yra poreikis modeliui suteikti informaciją, kuri yra visiškai nauja ir/ar nebuvo naudota pirmio treniravimo metu. Atsiradus „*chatGPT*“ platformai buvo pastebėta, jog turimi modeliai sugeba mokytis ir atlikti užduotis iš pateikto konteksto (angl. *in context learning*), ko pasekoje išpopuliarėjo RAG metodo taikymas. Pačioje paprasčiausioje šio metodo formoje įprastai atliekami šie žingsniai:

1. **Indeksavimas** Norimi duomenys išvalomi ir konvertuojami į paprastą tekstą. Tekstas suskaidomas į mažesnius fragmentus (angl. *chunks*), kurie paverčiami į vektorius naudojantis įterpimo modeliu (angl. *embedding model*) ir patalpinami į vektorių formatą, palaikančią duomenų bazę
2. **Paieška** Į D.K.M. pateikta užklausa yra paverčiama vektoriais, naudojant tą patį modelį kaip indeksavimo metu. Tuomet yra apskaičiuojamas panašumo vertinimas tarp užklauskos vektoriaus ir indeksuotų fragmentų vektorių, dažniausiai tai yra kosinusinis panašumas (angl. *cosine similarity*) arba euklidinis atstumas. Nustačius panašumą yra išrenkami (angl. *Top K*) aukščiausią įvertinimą gavę fragmentai.
3. **Atsakymo generavimas** Atrinkti fragmentai sujungiami su pateikta užklausa sudarant vientisą D.K.M. įvestį. Tokiu būdu modelis pateikia atsakymą, remdamasis tiek žiniomis iš pradinio treniravimo, tiek pateikta nauja informacija.

Darbo citegao2023retrieval autoriai šį metodą vadina naiviu RAG ir teigia, jog papildomai galima pridėti žingsnių, kurie patobulintų kiekvieną iš pagrindinių sudedamųjų dalių, bet šio darbo metu bus aptariami tik principiniai šio metodo elementai bei taikymas. Tyrimo [JSL⁺24] autoriai savo kuriamoje visuomenės simuliacijoje naudoja RAG metodą kaip pagalbinį įrankį, kuriant agentų atmintį. Jo pagalba agentai turi funkcionalumą atsižvelgti į paieškos etape surastus įvykius ir esamą situaciją planuojant savo veiksmą. Savitą, principinę RAG metodo dalį naudojančią, implementacijos funkciją aprašo ir darbo [POC⁺23] autoriai, bet kartu su teksto vertimu į vektorius ir jų palyginimo yra naudojami ir kiti panašumo rodikliai gaunami naudojantis D.K.M. skirtų žinučių inžinerija (angl. *prompt engineering*) taip efektyviai kombinuojant aptartus metodus.

3.4.2. D.K.M. papildomas treniravimas (angl. *angl. Finetuning*)

Kitas metodas, kurį darbo [ALR⁺25] autoriai įvardija kaip galimą sprendimą pritaikyti bendros paskirties D.K.M. visuomenės simuliacijoms, yra modelio papildomas treniravimas (angl. *finetuning*). Šis metodas priešingai negu RAG, pakeičia esamo D.K.M. sluoksnius ir parametrus prieš jo implementaciją į sistemą. Remiantis darbu [PZK⁺24], papildomas treniravimas yra vykdomas naudojantis baziniu modeliu (angl. *base model*), kuriam jau buvo atliktas pirminis treniravimas (angl. *pre-training*). Plačiąja prasme šis metodas pasinaudoja bazinio modelio turimomis žiniomis ir jas papildo ar pritaiko naujoms užduotims. Nors jį galima įvairiai klasifikuoti, paprasčiausia išskirti dvi alternatyvas:

- **Treniravimas su žymėtais duomenimis** Treniravimui naudojami duomenys yra pateikiami su žymomis, kurių tikslas modelį įgalinti atlikti naujas užduotis. Pavyzdžiui sukurti verslo asistentą, kuris sugebės klientams padėti naviguoti produkto sistemoje. Tokioje situacijoje duomenys

būtų sužymimi naudojantis verslo terminologija ir/ar galimais klientų klausimais, taip modeliui suteikiant daugiau žinių apie verslo aplinką, situacijas ir naudojamus žodžius.

- **Treniravimas su nežymėtais duomenimis** Šioje alternatyvoje duomenys yra nežymėti ir šis metodas dažniausiai yra naudojamas, kuomet duomenų įrašus yra sudėtinga sužymėti. Šis būdas irgi gali padėti praplėsti ar geriau pritaikyti bazinio modelio žinias, bet dažnu atveju modelio išvestis yra mažiau tiksli, lyginant su anksčiau aptarta alternatyva.

Remiantis darbo [PZK⁺24] autoriais papildomo treniravimo proceso pagrindinės sudedamosios dalys yra šios:

1. **Duomenų paruošimas** Pirmame etape paruošiamas duomenų rinkinys, kuris atitinką konkrečią užduotį kaip instrukcijų generavimą, nuotaikos analizę ar temų žymėjimą.
2. **Modelio paruošimas** Šiame etape yra parenkamas bazinis modelis ir užkraunami jo svoriai neretai nusakant kiek ir kokių sluoksnių modelio architektūroje yra.
3. **Treniravimo aplinkos paruošimas** Šiame etape paruošiami pradiniai nustatymai, apibrėžiantys treniravimosi eigą. Pagrindiniai iš jų yra: epochų/žingsnių skaičius, mokymosi greitis, gradiento variaciją valdantys parametrai, tikslumo funkcija bei kiti, priklausantys nuo papildomo treniravimo varianto.
4. **Pilnas ar dalinis treniravimas** Šio etapo metu vyksta treniravimas ir yra atnaujinami bazinio modelio sluoksniai ir jų svoriai. Priklausomai nuo pasirinkto varianto gali būti atnaujinami visi sluoksniai ar tik dalis jų su galimybe sukurti papildomus sluoksnius, kurie baigus treniravimą sujungiami su baziniu modeliu.
5. **Vertinimas** Treniravimo metu ir/ar po jo vykdomas modelio vertinimas siekiant apskaičiuoti jo tikslumą, dažnu atveju pasitelkiant validacijos duomenis, kurie nebuvo naudojami modelio treniravimo metu. Gaunama vertė yra lyginama su treniravimo metu gautu tikslumu ir vertinama ar modelis sėkmingai įsisavino informaciją ir tinkamai atnaujinio svorius.

Papildomo treniravimo metodą darbo [PTY⁺24] autoriai naudoja kaip galima sprendimą aptartoms D.K.M. modelių šališkumo bei žmogaus vertybių neatitikimo problemoms spręsti. Sukurtoje sistemoje pradinis D.K.M. naudojamas įvairioms rolėms ir jų socialinėms sąveikoms kurti, bei jas pačiam vertinti, tai yra sulyginama su veiksmu, kuomet vienas aktorius atlieka daug skirtingų vaidmenų (angl. *monopolylogue*). Tuomet gauti duomenys yra naudojami papildomam treniravimui atlikti, siekiant sukurti mažiau šališką ir geriau žmogaus vertybes atspindintį modelį. Straipsnio [PTY⁺24] autorius įvertinęs rezultatus teigia, jog jo papildomai treniruotas modelis vertinimuose yra pranašesnis už „GPT-4“.

3.4.3. Modelio įvesties modeliavimas (angl. *Prompt-engineering*)

Paskutinis metodas labai plačiai naudojamas tiek simuliacinių, tiek įvairių kitų, D.K.M. pagalba veikiančių, sistemų kūrimo, kuomet nustatytas įvesties/išvesties srautas netenkina užsibrėžto tikslo.

Anot straipsnio [Ama24] autorių, modeliavimo tikslas yra sukurti tokią įvestį, kuri padėtų modeliui sugeneruoti pačią geriausią išvestį nusistatytam tikslui. Dažniausiai manipuluojama vartotojo (angl. user prompt) bei sistemos įvestimi (angl. system prompt), pastaroji padeda nusakyti modelio ir/ar jo paties įsivaizduojamą vaidmenį, toną, elgseną ir atsakymų stilių viso pokalbio metu. Anot straipsnio [SIB⁺24] autorių, dažniausiai modeliuojama šiomis priemonėmis:

- **Instrukcija (angl. Directive)** Tai sakiny s ar keli sakiniai nusakantys ką modelis turi atlikti pvz.: „Pateik man penkis geriausius šių metų filmus.”
- **Pavyzdžiai** Tekstas, kuris naudojamas kaip pavyzdys ar pavyzdžiai, nurodantys kokios išvesties yra tikimasi pvz.: „Filmas(1): „Titanikas”, Filmas(2) „Tadas Blinda””
- **Išvesties formatas** Nurodymas modeliui formatuoti savo išvestį nurodytu formatu pvz.: „Savo išvestyje naudok TIK JSON formatą”
- **Stilius** Šalia struktūrinių nuorodų galima teikti ir stiliaus nuorodas pvz.: „Tekstą pateik nenuosekliai ir trumpais sakiniais.”
- **Rolė** Tekstas, nurodantis iš kokios roles ar personažo perspektyvos modelis pateiks išvestį pvz.: „Tu esi garsiausias filmų kritikas Lietuvoje.”
- **Papildoma informacija/kontekstas** Šis komponentas yra platus ir gali susidaryti iš bet kokios informacijos, kuri modeliui gali būti reikalinga generuojant išvestį, gali būti panaudojama ir su RAG sistema sugeneruoti konteksto blokai.

Skirtingais būdais naudojantis šiais komponentais galima sudaryti įvairius įvesties modeliavimo metodus, kurie yra taikomi pagal siektiną tikslą. Pagal straipsnį [SIB⁺24] galima išskirti pagrindines metodų grupes:

- **„Few-shot prompting”** Šio metodo pagrindinė sudedamoji dalis yra modeliui pateikiami pavyzdžiai, jie gali būti pateikiami sisteminėje įvestyje arba jungiami su vartotojo užklausa. Naudojantis šiuo metodu svarbiausia tinkamai pasirinkti pavyzdžių kiekį, formatą, panašumą ir prieš/po pavyzdžiais esančias instrukcijas.
- **„Zero-shot prompting”** Priešingai nuo praeito metodo, šiame modeliavime nėra pavyzdžių, bet gali būti kiti komponentai. Šiai klasei priklauso metodai, kurie gali atskirai išnaudoti rolės ir stiliaus komponentus siekiant kurti išvestį, paremtą žiniomis ir perspektyva iš nusakytos rolės ar generuoti tekstą norima stilistika.
- **„Thought Generation”** Šioje grupėje yra siekiama, jog modelis pateiktų savo protavimą ir generuoto teksto pagrindimą. Vienas populiariausių šios grupės metodų yra „Minčių grandinės” (angl. *chain-of-thought* metodas, kurį galima įgyvendinti prie vartotojo įvesties pridėdant sakinį: „Galvokime žingsnis po žingsnio”, tokiu būdu modeliui liepiant ne tik atsakyti į klausimą, bet ir argumentuoti savo „kelią” iki atsakymo.

- „**Ensembling**” Metodai esantys šioje grupėje įgyvendinami naudojant daug įvairių vartotojo ir sisteminių įvesčių, kurios po to skirtingais būdais sujungiamos į vieną išvestį. Kaip pavyzdį galima naudoti prieš tai aptartą metodų klasę „Few-shot prompting” ir kurti įvestis su daug skirtingų pavyzdžių, o juos įvertinus vartotojui pateikti vieną išvestį. Neretai tai yra efektyvus būdas paslėpti D.K.M haliucinacijas, bet sunaudojant daugiau resursų.

Šiose metodų klasėse yra daug variacijų ir kombinacijų, ko pasekoje, D.K.M. pagalbą implemenuotų sistemų autoriams gali būti sunku iš karto pasirinkti tinkamą jų naudojimą. Pavyzdžiui, kurdami simuliacinę sistemą, darbo [POC⁺23] autoriai naudoja kelis įvesties modeliavimo komponentus kaip instrukciją, papildomą kontekstą bei formato nurodymus, matomus 2 iliustracijoje. Šie įrankiai yra pasitelkiami siekiant gauti D.K.M. sugeneruotą vertinimą apie simuliuojamo agento atminties įrašus.

```
On the scale of 1 to 10, where 1 is purely mundane
(e.g., brushing teeth, making bed) and 10 is
extremely poignant (e.g., a break up, college
acceptance), rate the likely poignancy of the
following piece of memory.
Memory: buying groceries at The Willows Market
and Pharmacy
Rating: <fill in>
```

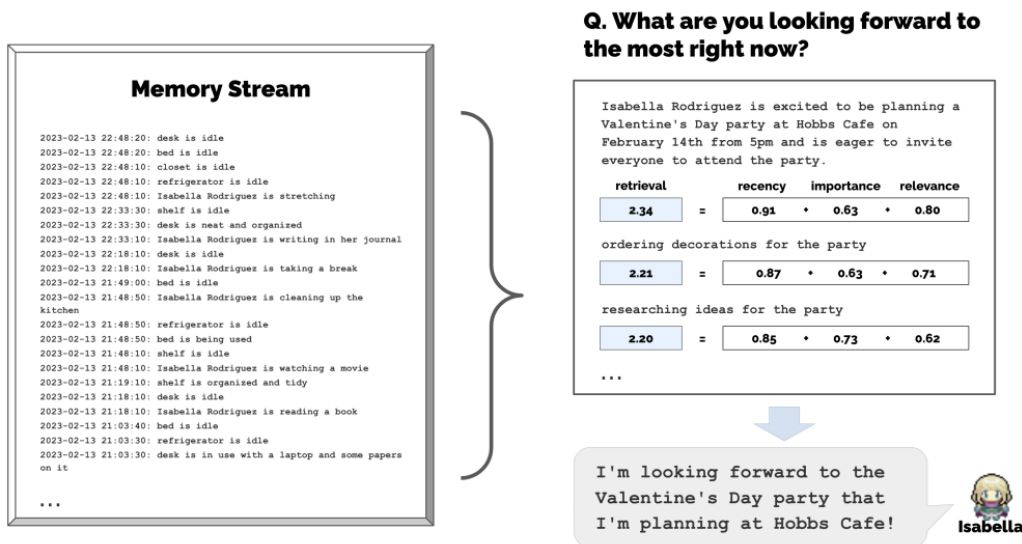
2 pav. Įvesties modeliavimo iliustracija [POC⁺23]

4. Įgyvendinti pavyzdžiai

4.1. Mokslinis tyrimas „*Interaktyvus žmonių elgesio atspindys*“

Darbo [POC⁺23] tikslas yra ištirti aplinką, sudarytą iš 25 agentų, kurie veikia D.K.M. ir kitų algoritminių veiksmų pagalbą kaip įrankį, kuris padėtų „repetuoti“ ir/ar maketuoti įvairias socialinių santykių situacijas. Aplinka yra panaši į žaidimą „*Sims*“, bet priešingai nuo jo, sistemoje veikiantys agentai veikia autonomiškai ir sistemos sukurtą aplinką stebintis vartotojas jų valdyti negali. Kiekvienas iš agentų turi savo dienos rutiną, namus, darbą, socialinį ratą bei atmintį ir refleksijas. Išsiaiškinti ar šie komponentai įgalina agentus generuoti veiksmus panašius į realaus žmogus, buvo pasitelkti du metodai. Vienu atveju agentas gyveno „izoliacijoje“, kitu atveju du agentai gyveno ir komunikavo kartu, dvi simuliacijoje nustatytas dienas. Agentai buvo apklausiami ir tikrinama, ar jie sugeba išlaikyti savo charakterį ir tikslingai atsiminti faktus, naudojantis savo atmintimi. Viena problematiškiausių vietų buvo agentų tendencija pamiršti faktus, juos pagražinti ar kalbėti/elgtis pernelyg formaliai. Nepaisant to buvo prieita prie išvados, jog tokia aplinka yra įmanoma ir agentai sugeba įtikinamai simuliuoti žmogaus veiksmus, kaupti patirtį, bei formuoti gilesnį supratimą apie save ir aplinką, atliekant refleksiją.

Šio darbo autoriai nemažai dėmesio skyrė kiekvieno iš aptartų simuliacijos architektūros elementų implementacijai todėl galima paanalizuoti kaip anksčiau nagrinėti metodai atsispindi praktinėse implementacijose. Darbo eigoje viena pagrindinių technologinių problema, kurią sprendė darbo [POC⁺23] autoriai buvo dabartinių D.K.M. konteksto ilgio apribojimai, kurie neleido visai agento atminčiai tilpti D.K.M. įvesties režimuose. Todėl autoriai sukūrė atminties saugojimo sistemą, kurioje buvo saugomi įvairūs įvykių aprašymai, jų laikas ir informacija apie paskutinį kartą, kuomet šis įrašas buvo panaudotas. Dirbti su šia sistema buvo sukurta „*retrieval*“ funkcija, kuri kaip įvestis naudojo esama agento situaciją, o išvestimi tapo atminties objektų sąrašas, naudojamas kaip įvestis į D.K.M. taip efektyviau išnaudojant modelio palaikomą konteksto ilgį. Šios funkcijos implementacija buvo įvairialypė, priklausanti nuo to, kas yra svarbu kiekvienam iš agentų. Atminties objektai matomi paveikslėlyje 3 buvo vertinami pagal 3 kriterijus: naujumą - kaip senai paskutinį kartą buvo naudojamas objektas, tuomet objekto svarbumą, pavyzdžiui pusryčių valgymas šioje sistemoje mažiau svarbus už vestuves, ir galiausiai aktualumą, parodantį kiek objektas atitinka esamą situaciją.



3 pav. Atminties srauto vizualizacija [POC⁺ 23]

Svarbumas buvo vertinamas modeliui pateikiant užklausą su klausimų įvertinti situaciją dešimbalėje skalėje pateikiant pavyzdžius, kas atitiktų žemiausią ir aukščiausią vertinimus, tokiu būdu pasinaudojant aptartu įvesties modeliavimo metodu. O aktualumas buvo vertinamas paverčiant atminties objektus ir esamą situaciją įterpinių vektoriais (angl. *Embedding vectors*) ir suskaičiuojant kosinusinį panašumą (angl. *cosine similarity*) tarp jų, naudojant RAG metodo pagrindiniu principu. Galiausiai šių vertinimų reikšmės buvo normalizuojamos intervale [0, 1] ir sudedamos padauginus kiekvieną jų iš daugiklio pagal tai kiek daug dėmesio norima skirti kiekvienam iš kriterijų. Galutinė formulė atrodo taip:

$$\text{įvertinimas} = \alpha_{\text{naujumas}} \cdot \text{naujumas} + \alpha_{\text{svarbumas}} \cdot \text{svarbumas} + \alpha_{\text{aktualumas}} \cdot \text{aktualumas}$$

Kitas iššūkis buvo refleksijos sistema. Refleksijos buvo laikomos kartu su kitais prisiminimais. Jos susidarydavo kelis kartus per sistemos nustatytą dieną, surenkant 100 paskutinių prisiminimų ir užklauskiant D.K.M. kokius tris svarbiausius klausimus gali iš jų susidaryti apie agentą. Šie klausimai buvo naudojami kartu su „*retrieval*“ funkcija siekiant surasti prisiminimus pagal šiuos klausimus, kurie galiausiai buvo panaudojami kaip nauja įvestis į D.K.M. taip kuriant naujas įžvalgas ir formuojant refleksiją. Svarbu paminėti, jog šiame procese refleksijos gimsta ne tik iš prisiminimų, bet ir iš kitų refleksijų, tokiu būdu praplečiant refleksijos svarbą.

Dar viena svarbi problema, kurią autoriai sprendė, buvo agento dienos planavimas. Jis buvo įgyvendinamas užklauskiant D.K.M. atskirai apie kiekvieną įvykį, bet sistemos pradžioje buvo gaunama labai daug nelogiškų bei pasikartojančių veiksmų, pavyzdžiui, agentas valgydavo pietus tris kartus per dieną. Todėl planą buvo pradėta kurti principu „iš viršaus į apačią“, pradedant nuo to, jog dienos pradžioje buvo užklausiama abstraktaus visos dienos plano, o vėliau pakartojant užklauskas kas valandą, o galiausiai 15 min intervalais, kas papildomai leido kartu ir plačiau aprašyti veiklą ir suteikti daugiau detalių bei įdomumo. Šioms planų iteracijoms kurti D.K.M. įvesčiai buvo naudojamas agento aprašas su jo charakterio apibūdinimu, neseniai išsaugotomis patirtimis bei praėjusios dienos įvykiais. Veiksmų

iteracijos kartu su refleksijomis irgi buvo saugomos atminties sraute ir rekursyviai kviečiamos siekiant pridėti daugiau detalių. Sistemoje veiksmai veikė ciklu, kuriame po kiekvieno atlikto veiksmo agentas reaguodavo į aplinką ir rinkdavosi ar tęsti veiklą ar reaguoti į aplinką. Šiam sprendimui priimti D.K.M. kaip įvestis buvo naudojama užklausa su agento aplinkos vertinimu bei aktualiais prisiminimais, o išvestimi tapdavo sprendimas ir galimos reakcijos detalus aprašymas.

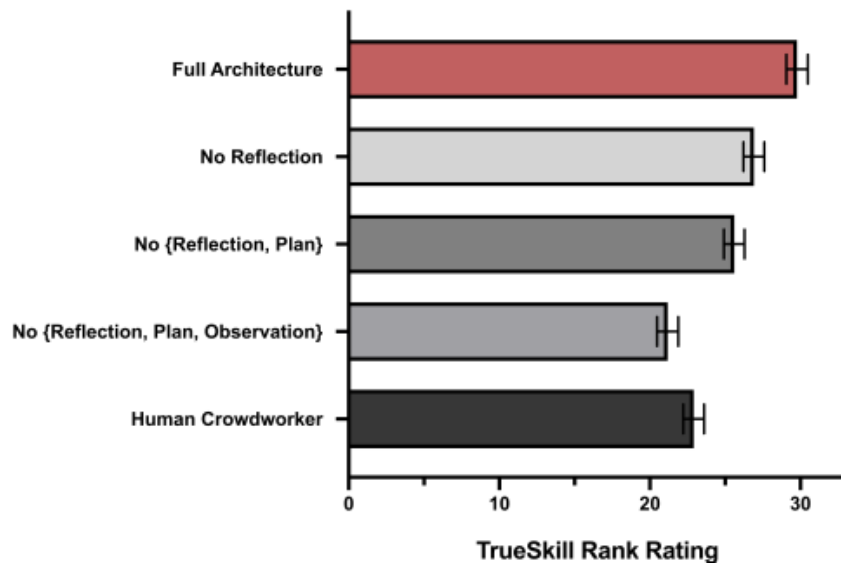
4.1.1. Sukurtos sistemos vertinimas

Moksliniame darbe [POC⁺23] autoriai vertino savo sukurtą sistemą naudodami dvi pagrindines metodikas: kontroliuojamą eksperimentą ir visapusišką (angl. *end-to-end*) vertinimą.

- **Kontroliuojamas eksperimentas** Šiame etape sistemai vertinti buvo pasitelktą 100 žmonių, kurie vertino kiekvieno agento atlikto interviu tikroviškumą penkiose kategorijose: suvokimas apie save, atmintis, planavimas, reakcijos ir refleksijos. Buvo vertinamos kelios architektūros versijos, su pasirinktinai panaikintais elementais kaip pavyzdžiui architektūra be refleksijos ir planavimo, taip siekiant nustatyti, kurie elementai labiausiai prisideda prie sistemos kokybės. Rezultatai buvo pateikiami šachmatų reitingo sistemose naudojama „TrueSkill“ skalėje. Apibendrinant gautus rezultatus matomus 4 diagramoje, galima pastebėti, jog vertinimas buvo mažiausias, kuomet iš architektūros buvo panaikinti refleksijos, planavimo bei aplinkos stebėjimo elementai, o sistema su visais elementais gavo didžiausią vertinimą, iš ko galima teigti, jog visi jie yra svarbūs ir efektyvūs. Svarbu papildomai paminėti, jog šio eksperimento metu buvo pastebėta, jog dažniausiai pastebima problema buvo aptartas iš D.K.M. kylantis agento polinkis pagražinti faktus iš savo atminties srauto, ką darbo autoriai įvardija kaip modelio haliucinaciją.
- **Visapusiškas vertinimas (angl. *end-to-end*)** Šio testo metu buvo testuojama architektūra su visais joje esančiais simuliacijos elementais, kuriant scenarijų, kuriame 25 agentai išgyveno dvi pilnas dienas jiems sukurtame miestelyje. Scenarijaus pradžioje vienam iš agentų buvo suteikta informacija su užduotimi suorganizuoti Valentino dienos šventę, o kitam faktas, jog jis balotiruojasi tapti miestelio meru. Praėjus dviems sistemoje nustatytoms dienoms, buvo apklausti visi agentai siekiant nustatyti ar juos pasiekė ši informacija apie pradines užduotis. Vertinant jų pateiktus atsakymus kaip taip arba ne, buvo pastebėta, jog trylika agentų žinojo apie Valentino dienos šventę, o aštuoni apie agento kandidatūrą į mero poziciją. Patikrinus simuliacijos istoriją buvo nustatyta, jog visi agentai gavę šią informaciją jos neišsigalvojo ir sužinojo per įvykusias sąveikas. Papildomai buvo pastebėta, jog agentai sugebėjo dieną prieš šventę imituoti pasiruošimą. O vertinant visas tarp agentų įvykusias interakcijas buvo pastebėta, jog tik 1.3% iš 453 sugeneruotų veiksmų ir aplinkos stebėjimų buvo klasifikuoti kaip haliucinacija.

Šis tyrimas atskleidė ir aptartas D.K.M. problemas. Viena iš jų buvo tai, jog didinant atmintį, *retrieval* funkcija ne visais atvejais sugebėdavo parinkti logiškus veiksmus. Papildomai buvo susidurta ir su logikos problema, kuomet agentai naudodavosi tuo pačiu objektu vienu metu ar pavyzdžiui eidavo į parduotuvę, kuomet ji jau buvo uždaryta. Paskutinė pastebėta problema buvo pernelyg didelis formalumo lygis pokalbiuose, kuris neretai lemdavo tai, jog agentai

sutikdavo su visomis jiems siūlomomis veiklomis, net jei jos prieštaraudavo jų charakterio bruožams.



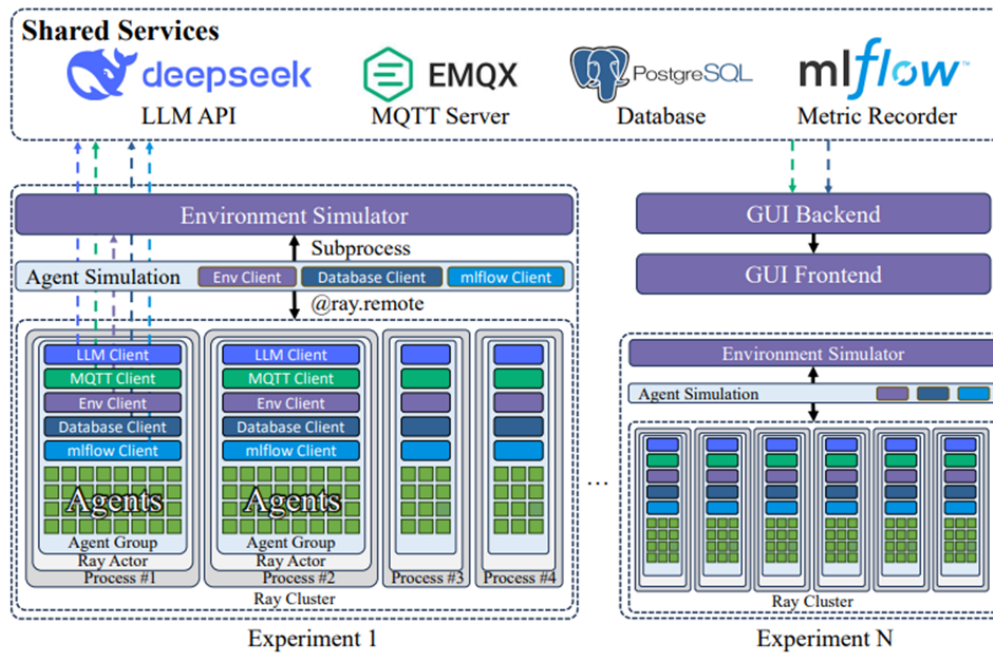
4 pav. Kontroliuojamo eksperimento rezultatų diagrama [POC⁺23]

4.2. Mokslinis tyrimas „Agentų visuomenė“

Darbo [PYZ⁺25] tikslas buvo sukurti didelės apimties „socialinį simulatorių“ D.K.M agentų pagalba. Jo maksimali pasiekta apimtis buvo 10 tūkstančių agentų ir 5 milijonai veiksmų tarp agentų, bei tarp agentų ir jų aplinkos. Simuliacijos pagalba buvo siekiama sukurti aplinką, kurią būtų galima naudoti socialiniams dėsniams apskaičiuoti bei įvertinti. Buvo gilinamasi į keturias socialines problemas: poliarizaciją, neapykantą kurstančios informacijos sklaidą, politinių sprendimų poveikį ir išorinių visuomenės sukrėtimų rezultatus kaip pvz. uraganas. Pagrįsti teisingą simuliacijos veikimą buvo pasitelkta informacija iš realaus pasaulio tyrimų apie šias keturias problemas. Buvo lyginamos priežastys, dėsningumas ir vidiniai socialinių problemų mechanizmai abejose aplinkose. Įvertinus rezultatus, darbo autoriai teigė, kad ši ir panašios simuliacinės aplinkos gali tapti efektyviais įrankiais socialiniuose moksluose, kurie padėtų su mažesniais finansiniais kaštais ir rizika įvertinti įvairius politinius sprendimus, socialines krizes, sekti ekstremalių ideologijų plitimą ir poliarizaciją, bei įvertinti galimų sprendimų efektyvumą.

Šio darbo analizė padeda suvokti su kokiomis problemomis susiduria simuliacinės sistemos, kurios pasižymi didele apimtimi bei kompleksiškais tikslais, kuriais siekiama tapti realiu sociologijoje naudojamu tyrimų įrankiu. Straipsnio [PYZ⁺25] autoriai siekė sukurti simuliaciją, kuri galėtų plėstis ir palaikyti itin didelį agentų bei sąveikos tarp jų mastą todėl susidūrė su didelių kompiuterinių resursų valdymo problemomis. Papildomai autoriai siekė įgyvendinti lygiagretų agentų veikimą priešingai nuo kitų simuliacijų, kur agentai veikia nuosekliame cikle. Šioms problemoms spręsti buvo naudojamos atviro kodo bibliotekos, kurios anot autorių buvo pasiekusios užtektinai didelį brandumo lygį. Simuliacija buvo suskirstyta į kelis pagrindinius komponentus matomus 5 diagramoje.

5.2 System Architecture



5 pav. Atminties srauto diagrama [PYZ⁺ 25]

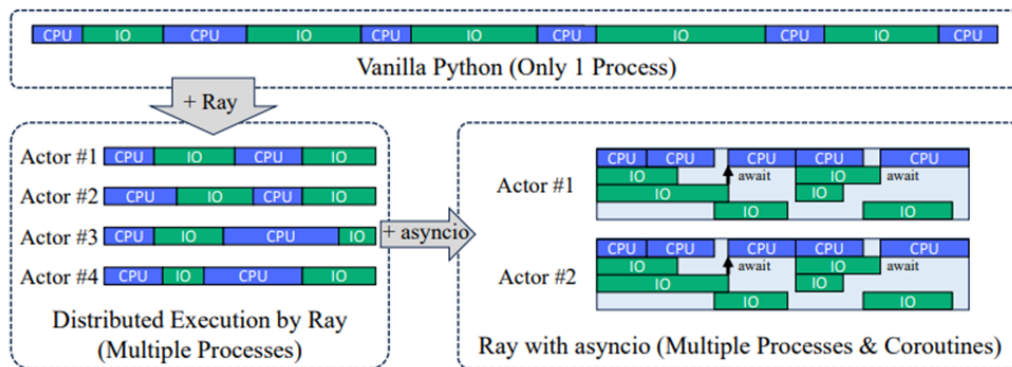
Kaip viena pagrindinių sudedamųjų dalių yra išskiriami bendri resursai:

- **D.K.M sąsaja** - tyrimo metu naudojami „OpenAI“ ir „DeepSeek“ modeliai.
- **„MQTT“ protokolo serveris** - šis serveris buvo naudojamas siųsti žinutėms tarp skirtingų agentų. Šis protokolas dažnai interneto erdvėje (angl. IoT) dėl savo ypatumo palaikyti didelius kiekius žinučių su sąlyginai mažais resursų kaštais.
- **Duomenų bazė** - šio tyrimo metu buvo naudojama „PostgreSQL“ duomenų bazė kaupti informacijai apie simuliacijos rezultatus tolesniam vertinimui. Ji buvo pasirinkta dėl efektyvaus, į grupės suskirstytų, duomenų rašymo, kuris buvo reikalingas su tokio plataus masto simuliacija.
- **Metrikų rinkimo sistema** - Darbo autoriai pasirinko naudoti „MLflow“ biblioteka, kuri palengvino statistinių duomenų rinkimą ir vertinimą.

Kaip paskutinis, bendras resursas, veikiantis atskirai nuo agentų, buvo aplinka su kuria sąveikavo tiek agentai, tiek viršuje nurodyti, kiti bendri resursai.

Likę objektai buvo atskiri agentų klasteriai bei grafinė sąsaja. Efektyviai valdyti didelį agentų kiekį buvo naudojama „Ray“ architektūra bei „Python asyncio“ biblioteka. Šios architektūros pagalba agentai buvo suskirstomi į grupes, kurios veikė kaip atskiri procesai, kas leido sumažinti TCP protokolui reikalingų lizdų kiekį, sutraukiant grupės agentų siunčiamas užklausas į vieną, jog nebūtų viršytas „MQTT“ palaikomų lizdų kiekis – 65535. Kiekvienam procesui buvo skirtas bendrų resursų klientas: D.K.M. sąsajos, „MQTT“ serverio, duomenų bazės, aplinkos bei metrikų rinkimo sistemos. Kaip galima

pastebėti 6 diagramoje, sistemoje buvo išlygiagriatinamos įvesties/išvesties operacijos reikalingos bendravimui su D.K.M sąsaja bei aplinka ir procesoriaus operacijos, reikalingos skaičiavimams agentų klasterių viduje, tokiu būdu sumažinant laukimo laiką.



6 pav. Agentų klasterio procesų išlygiagretinimo diagrama [PYZ⁺ 25]

Svarbu išnagrinėti ir *MQTT* serverio žinučių sistema, kuri leidžia simuliacijai tiksliau atvaizduoti realaus pasaulio komunikaciją ir pastebėti įvairius socialinius dėsningumus. *MQTT* yra naudojamas įgalinti komunikaciją tarp milijonų prie interneto prisijungusių įrenginių ir valdymo centrų naudojantis publikavimo ir prenumeravimo architektūra. Kiekvienas agentas turi savo unikalų ID, kurio pagalba vieni agentai gali publikuoti žinutes nustatytiems ID, o prenumeratoriai gauti žinutes iš nurodyto ID. Tokiu pat principu buvo vykdomas ir simuliacijos vertinimas - agentams iš vartotojo išsiunčiant apklausas, kurioms agentai pateikdavo struktūrizuotą atsakymą bei taikant interviu formatą, kuomet agentai pateikdavo nestruktūrizuotą atsakymą.

Papildomai darbo autoriai sukūrė nemažai įrankių, kurie palengvino simuliacijos sistemos kūrimą ir naudojimą, tokių kaip:

- **D.K.M sąsajos adapteris** - Biblioteka, kurios pagalba simuliacijoje galima naudoti įvairius modelius nepriklausomai nuo jų tiekėjo. Pavyzdžiui „OpenAI“, „DeepSeek“ ar „Qwen“.
- **D.K.M sąsajos perkrovimas** - Simuliacijoje yra funkcionalumas pakartotinai siųsti užklausą per sąsają jei ji nebuvo sėkminga.
- **JSON formato transliavimo sistema** - Sistemoje užklausos ir atsakymai D.K.M formatuojami *JSON* formatu, o šis įrankis konvertuoja juos į *Python* kalbos objektus paprastesniam jų panaudojimui. Šiuo metu populiarieji viešai prieinama *Python* biblioteka *Pydantic* irgi fasilituoja šios problemos sprendimą jei nėra poreikio kurti savo įrankį.
- **Grafinio atvaizdavimo platforma** - Šios platformos pagalba simuliacijos vartotojai gali realiu laiku stebėti agentų veiklą, siųsti jiems apklausas bei kartoti ar atsukti laiką eksperimentuose. Papildomai šis įrankis įgalina naudotoją vertinti duomenų bazėje kaupiamus įrašus apie agentus.

4.2.1. Sukurtos sistemos vertinimas

Įvertinti sistemos efektyvumui buvo pasitelkiami, autorių nuomone, esminiai klausimai:

1. Koks yra simuliacinės aplinkos implementacijos efektyvumas?

Įvertinti aplinkos efektyvumą buvo naudojamas kintantis agentų skaičius nuo 1000 iki 100000 ir vertinimas simuliacijos žingsnio laikas bei kiek užklausų per sekundę įvyksta aplinkoje. Kaip matoma ?? lentelėje, keičiant agentų kiekį buvo pastebėta, jog laikas bei užklausų kiekis pakito itin mažai, kas indikuoja, jog aplinka geba palaikyti itin didelį kiekį agentų su labai panašiu efektyvumu.

Table 2: Mean time per step with different numbers of agents.

# of Agents	Mean Time per Step (s) $\pm$ SD
10^3	$8.578 \times 10^{-3} \pm 3.0 \times 10^{-5}$
10^4	$9.129 \times 10^{-3} \pm 1.5 \times 10^{-5}$
10^5	$1.800 \times 10^{-2} \pm 5.66 \times 10^{-4}$
10^6	$0.1680 \pm 5.34 \times 10^{-4}$

7 pav. Simuliacijos aplinkos užklausų laiko lentelė [PYZ⁺25]

2. Koks yra „MQTT“ serverio žinučių sistemos efektyvumas lyginant su alternatyviomis implementacijomis?

Įvertinti „MQTT“ žinučių sistema buvo pasitelkta 100000 agentų ir 100 baitų ilgio žinutė, kuri buvo atsitiktinai išsiųsta dešimčiai kitų agentų. Taip pat buvo keičiamas lygiagrečių procesų skaičius nuo 4 iki 32. „MQTT“ protokolas buvo lyginamas su „Redis Pub/Sub“ sistema, „RabbitMQ AMQP“ protokolu bei „Kafka“ sistema. Kaip pagrindinis vertinimo kriterijus buvo pasirinktas persiunčiamų žinučių per sekundę skaičius 5 minučių intervale. Kaip minimalus reikalavimas buvo įvestas rodiklis reikalaujantis 20000 žinučių per sekundę. Sistema „Kafka“ nesugebėjo per 5 minutes inicializuoti 10000 agentų todėl buvo atmesta, o kaip matoma 8 paveikslėlyje, „RabbitMQ“ vos išpildė minimalų reikalavimą su 23000 žinučių. Galiausiai „Redis Pub/Sub“ sistema palaikė dvigubą žinučių kiekį lyginant su „MQTT“ nepaisant to, ji nebuvo pasirinkta, nes nepalaikė grafinio atvaizdavimo įrankių reikalingų simuliacijos stebėjimui, klaidų taisymui ir testavimui, todėl galiausiai buvo pasirinktas „MQTT“ variantas.

Table 3: Comparison of different messaging systems.

System	Best Parallel Process Number	Throughput (msg/s)	Auxiliary Tools
MQTT (emqx v5.8.1)	32	$44,702.1 \pm 111.3$	Built-in GUI
Redis Pub/Sub (v6.2)	16	$81,216.2 \pm 333.6$	-
RabbitMQ (v4.0.5)	16	$23,667.3 \pm 1,777.7$	Built-in GUI

8 pav. Skirtingų žinučių sistemų įvertinimų lentelė [PYZ⁺25]

3. Koks yra didelio masto socialinės simuliacijos su išvardintais komponentais efektyvumas?

Vertinant visos sistemos implementacijos efektyvumą buvo pasitelkiama nuo 1000 iki 10000

agentų bei skirtingas procesų kiekis tarp 8 ir 32. Toliau buvo vertinama kiek laiko užtrunka visiems agentams įgyvendinti 5 interakcijas, kiek įvesties žetonų (angl. *tokens*) reikalauja visos išvesties ir įvesties užklausa, D.K.M. sąsajos užklausoje patenkinimo laikas bei aplinkos užklausoje laikas. Rezultatuose pavaizduotose 9 lentelėje galima įžvelgti, jog kiekvieno simuliacijos etapo bendras laikas 5-tame stulpelyje mažėjo didėjant agentų kiekiui, kas indikuoja, jog sistemos išlygiagretinimo sprendimai veikia efektyviai. Nepaisant to didžiąją laiko dalį sudarė išorinės D.K.M. sąsajos užklausa, kurios yra neprognozuojamas ir apriboja tolimesnę tokių simuliacijų sistemų plėtrą. Darbo [POC⁺23] autoriai kaip sprendimą siūlo lokalių modelių alternatyvą, bet norint ją įgyvendinti reikėtų daug didesnių pradinių išlaidų vaizdo plokštėms bei modelio konfigūracijoms implementuoti.

Table 4: Performance metrics for different configurations.

#Agents	#Groups	Parameters			Average Time Cost		
		LLM Calls	ITs (/call)	OTs (/call)	All (s/round)	LLM (s/call)	Env (ms/call)
10 ³	8	4803.0	430.04	79.17	82.45	4.51	12.26
10 ³	16	3120.8	398.78	77.18	41.17	2.92	14.31
10 ³	32	4790.4	412.82	75.56	43.30	2.94	9.55
10 ⁴	8	54135.4	430.35	75.84	5681.18	52.54	33.55
10 ⁴	16	54002.2	430.24	75.80	1422.48	3.53	33.55
10 ⁴	32	54075.0	430.47	76.14	458.82	8.05	30.53

9 pav. Skirtingų žinučių sistemų įvertinimų lentelė [PYZ⁺25]

Išvados

Didžiųjų kalbos modelių taikymas visuomenės simuliacijos sistemose gali būti alternatyva arba priedas iki šiol simuliacijose taikytiems matematiniams modeliams. Be to, remiantis atlikta literatūros apžvalga, galima padaryti šias išvadas:

- D.K.M. pagalba įmanoma realizuoti architektūras, kuriose žmonės vaizduojami kaip agentai su atmintimi, vidine būseną ir gebėjimu autonomiškai priimti sprendimus pagal kontekstą bei aplinką.
- Simuliacinėse sistemose dažnai pasikartoja tos pačios problemos, kylančios tiek iš pačių D.K.M. (pvz., šališkumas, loginio nuoseklumo trūkumas), tiek iš sistemos architektūros (pvz., neefektyvi agentų struktūra).
- Norint pasiekti aukštą simuliacijos kokybę, būtina:
 - tinkamai išskaidyti sistemos sudedamąsias dalis ir jas įvertinti atskirai.
 - kryptingai taikyti metodus kaip RAG, papildomas treniravimas (angl. *finetuning*) ir įvesties modeliavimas (angl. *prompt engineering*).
- Svarbu atsakingai pasirinkti programines bibliotekas, skirtas agentų sąveikai, atsižvelgiant į:
 - norimą simuliacijos dydį
 - tiriamą scenarijų
 - vertinimo metodiką

Atsižvelgus į gautas išvadas, galima teigti, jog D.K.M. pagrindu grįstos visuomenės simuliacijos sistemos vis dar susiduria su nemažais technologiniais iššūkiais, bet jų potencialas socialinių mokslų tyrimuose yra labai didelis.

Literatūra ir šaltiniai

- [ALR⁺25] J. R. Anthis, R. Liu, S. M. Richardson, A. C. Kozlowski, B. Koch, J. Evans, E. Brynjolfsson, M. Bernstein. „LLM Social Simulations Are a Promising Research Method“. Iš: *arXiv preprint arXiv:2504.02234* (2025).
- [Ama24] X. Amatriain. „Prompt design and engineering: Introduction and advanced methods“. Iš: *arXiv preprint arXiv:2401.14423* (2024).
- [BS15] F. Bianchi, F. Squazzoni. „Agent-based models in sociology“. Iš: *Wiley Interdisciplinary Reviews: Computational Statistics* 7.4 (2015), puslapiai 284–306.
- [DP14] S. De Marchi, S. E. Page. „Agent-based models“. Iš: *Annual Review of political science* 17.1 (2014), puslapiai 1–20.
- [GXG⁺23] Y. Gao, Y. Xiong, X. Gao, K. Jia ir kiti. „Retrieval-augmented generation for large language models: A survey“. Iš: *arXiv preprint arXiv:2312.10997* 2 (2023), puslapis 1.
- [HZY⁺24] S. Han, Q. Zhang, Y. Yao, W. Jin, Z. Xu, C. He. „LLM multi-agent systems: Challenges and open problems“. Iš: *arXiv preprint arXiv:2402.03578* (2024).
- [JSL⁺24] Z. Jiang, Y. Shi, M. Li, H. Xiao, Y. Qin, Q. Wei, Y. Wang, Y. Zhang. „Casevo: A Cognitive Agents and Social Evolution Simulator“. Iš: *arXiv preprint arXiv:2412.19498* (2024).
- [KJA⁺24] D. Kumar, U. Jain, S. Agarwal, P. Harshangi. „Investigating Implicit Bias in Large Language Models: A Large-Scale Study of Over 50 LLMs“. Iš: *arXiv preprint arXiv:2410.12864* (2024).
- [MDH⁺24] X. Mou, X. Ding, Q. He, L. Wang ir kiti. „From Individual to Society: A Survey on Social Simulation Driven by Large Language Model-based Agents“. Iš: *arXiv preprint arXiv:2412.03563* (2024).
- [PYZ⁺25] J. Piao, Y. Yan, J. Zhang, N. Li ir kiti. „AgentSociety: Large-Scale Simulation of LLM-Driven Generative Agents Advances Understanding of Human Behaviors and Society“. Iš: *arXiv preprint arXiv:2502.08691* (2025).
- [POC⁺23] J. S. Park, J. O’Brien, C. J. Cai, M. R. Morris, P. Liang, M. S. Bernstein. „Generative agents: Interactive simulacra of human behavior“. Iš: *Proceedings of the 36th annual acm symposium on user interface software and technology*. 2023, puslapiai 1–22.
- [PS24] F. Perez-Cruz, H. S. Shin. *Testing the cognitive limits of large language models*. Techninė ataskaita. Bank for International Settlements, 2024.
- [PTY⁺24] X. Pang, S. Tang, R. Ye, Y. Xiong, B. Zhang, Y. Wang, S. Chen. „Self-alignment of large language models via monopolylogue-based social scene simulation“. Iš: *arXiv preprint arXiv:2402.05699* (2024).

- [PZK⁺24] V. B. Parthasarathy, A. Zafar, A. Khan, A. Shahid. „The ultimate guide to gurene-tuning llms from basics to breakthroughs: An exhaustive review of technologies, rese-arch, best practices, applied research challenges and opportunities“. Iš: *arXiv preprint arXiv:2408.13296* (2024).
- [SIB⁺24] S. Schulhoff, M. Ilie, N. Balepur, K. Kahadze ir kiti. „The prompt report: A systematic survey of prompting techniques“. Iš: *arXiv preprint arXiv:2406.06608* 5 (2024).
- [SJE14] F. Squazzoni, W. Jager, B. Edmonds. „Social simulation in the social sciences: A brief over-view“. Iš: *Social Science Computer Review* 32.3 (2014), puslapiai 279–294.
- [TN23] Y. Talebirad, A. Nadiri. „Multi-agent collaboration: Harnessing the power of intelligent llm agents“. Iš: *arXiv preprint arXiv:2306.03314* (2023).
- [WWT⁺25] Q. Wang, J. Wu, Z. Tang, B. Luo, N. Chen, W. Chen, B. He. „What Limits LLM-based Human Simulation: LLMs or Our Design?“ Iš: *arXiv preprint arXiv:2501.08579* (2025).